

Chapter 11: Legacy Monolith Refactoring & Break-Fix Engineering Lab

Brownfield vs. Greenfield In school and online tutorials, you always start with an empty `main.py`. In an enterprise role, you will almost never touch an empty file. You will inherit a 1,500-line function written four years ago by an engineer who left the company, full of mutable defaults, hidden global mutations, unhandled exception swallows, and zero unit tests.

This lab teaches you the principal engineer's methodology for refactoring hazardous legacy code into pristine, testable, modern Python without causing a production outage.

1. The Hazardous Legacy Codebase

Examine this order processing script taken from an actual real-world legacy code pattern:



2. The Golden Rule: Characterization Tests First

CAUTION

Never refactor code without a safety net. Before altering a single character of production code, write **Characterization Tests** (also called "Pinning Tests"). The goal is not to test what the code *should* do, but to lock down what it *currently* does, bugs and all, so any behavioral change is immediately flagged.

Writing the Pinning Harness with `pytest`



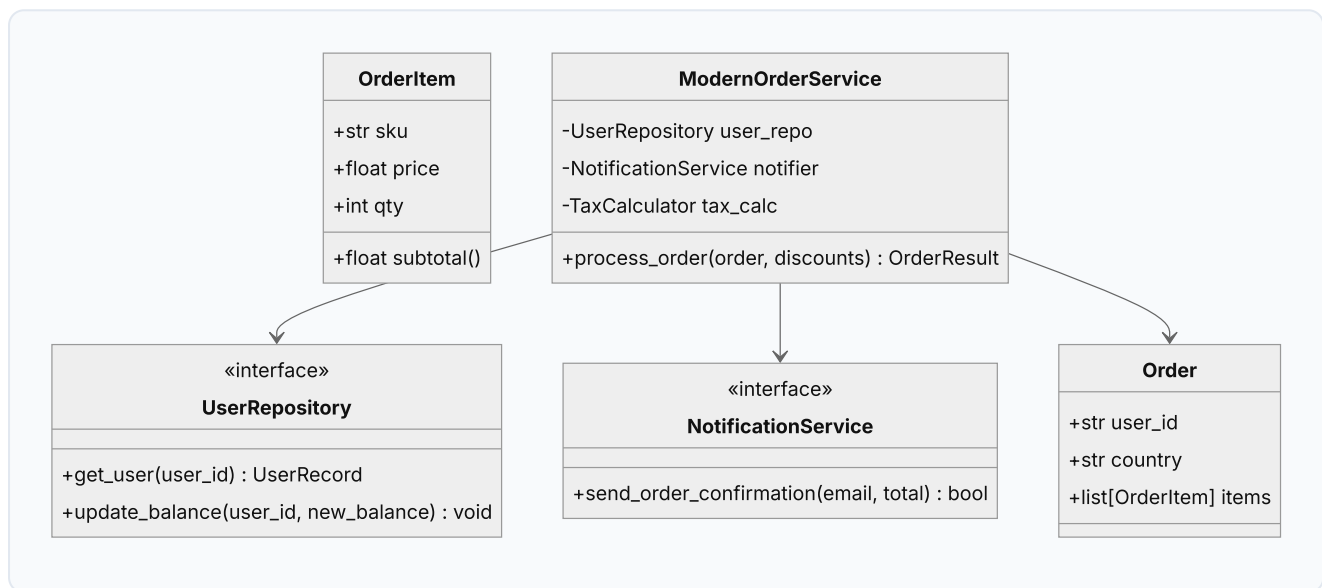
Run tests to guarantee our baseline passes:



3. Surgical Modernization Step-by-Step

We systematically dismantle the architectural anti-patterns:

1. **Replace string SQL with Parameterized Queries** (Eliminates SQL Injection).
2. **Eliminate Global State** (`GLOBAL_TAX_CACHE` and counters are injected via configuration).
3. **Fix Mutable Defaults** (Use `None` and initialize inside `__init__`).
4. **Decouple External Dependencies with Protocols** (Isolate SMTP Email and Database behind Abstract Interfaces).
5. **Model Domain with Dataclasses and Enums** (Replace loose dictionaries with type-safe structures).



4. The Pristine Refactored Implementation

5. Comparative Architectural Review

Dimension	Legacy Implementation	Modern Refactored Architecture
State Management	Polluted global tax cache, shared mutable list default	Fully encapsulated, immutable dataclasses, explicit dependency injection
Arithmetic Precision	Binary <code>float</code> (introduces IEEE-754 precision bugs)	High-precision <code>Decimal</code> with explicit currency quantization
Security	Vulnerable to SQL injection via format strings	Isolated behind Repository with parameterized query abstractions
Testability	Requires live SQLite file and real SMTP server	100% testable in memory using mock/stub protocols in under 5ms
Observability	Swallowed errors (<code>except Exception: pass</code>)	Structured logging with contextual JSON metadata (<code>structlog</code>)

Exercises

Three graded exercises for this chapter (two coding, one debugging) with hidden tests:



Attempt first; the reference solutions are in `exercises/_answers/ch11.py`.

Further Reading

- [Refactoring catalog \(Martin Fowler\)](#)
- [Strangler Fig pattern](#)
- [Characterization tests](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. Why write characterization tests before refactoring legacy code?

They pin down current behaviour so you can refactor safely and detect unintended changes.

▼ 2. What is the 'strangler fig' approach?

Route new behaviour to new code path by path while the old code keeps serving the rest, until the old code can be removed.

▼ **3.** What is a seam?

A place where you can alter behaviour without editing the code under test (dependency injection point, overridable method).

▼ **4.** Name two refactoring moves that reduce coupling.

Extract interface/Protocol and inject the dependency; move logic out of global state into a function taking parameters.